



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



Final Report: Twitter Sentiment Analysis for Product Feedback

CCBDA

Alicia García Sanz
Andreja Andrejic
Ignacio Lloret Lorente
Iker Jansà Estavillo
Martí La Rosa Ramos

Index

1. Introduction.....	2
2. Project Overview.....	2
3. Functionality and Scope.....	2
4. Architecture.....	3
4.1. Initial Architecture.....	3
4.1.1. Evolution of the Data Ingestion.....	4
4.1.2. Evolution of the Sentiment Analysis.....	4
4.1.3. Evolution of the Analytics.....	4
4.2. Final Architecture.....	5
4.2.1. Data Ingestion.....	5
4.2.2. Sentiment Analysis.....	6
4.2.3. Visualization and Analytics.....	7
5. System Process.....	7
5.1. Ingestion Pipeline.....	7
5.2. Prediction.....	8
5.3. Analytics.....	10
5.4. Visualization.....	11
6. Deployment.....	14
7. Use case.....	15
8. Project organization.....	16
8.1. Division of Responsibilities.....	16
8.2. Meetings and Planning.....	16
8.3. Time Allocation.....	17
9. Data Regulation.....	18
10. Twelve-Factor Methodology.....	19
11. Future Work.....	20
12. Conclusion.....	21
12.1 Challenges and Adaptations.....	21
12.2 Key Learnings.....	22

1. Introduction

In the digital age, social media platforms like Twitter have become central to understanding public opinion on a variety of topics, including consumer products. The real-time nature of Twitter, combined with its vast user base, makes it an invaluable resource for measuring consumer sentiment and reactions. Consumers frequently share their opinions, experiences, and feedback on products, offering companies a direct line to customer thoughts and feelings that can be more immediate and straightforward than traditional market research methods.

Our project, conducted by Team 7 of the CCBDA (Cloud Computing for Big Data Analytics) course, aims to harness this wealth of information by leveraging Twitter data to perform sentiment analysis on consumer feedback for specific products. By doing so, we can provide businesses with insights into how their products are perceived post-launch. This analysis not only captures general sentiment but can also drill down into specific aspects or features of a product that are driving consumer opinions, whether positive or negative.

By conducting this project, we seek to demonstrate the practical applications of big data analytics in consumer feedback analysis, showcasing how social media data can be transformed into valuable business intelligence.

2. Project Overview

The primary goal of our project is to analyze sentiments expressed in tweets about specific products post-launch, such as Nike shoes, Peugeot cars, or Nestle yogurts. By understanding public sentiment, companies can gain insights into consumer perceptions, providing information for future marketing strategies or product improvements. Additionally, companies may be interested in analyzing sentiments related to specific features, such as the screen size or color, of products like the iPhone. This granular analysis allows for a deeper understanding of consumer preferences and concerns, ultimately enabling the creation of products that better meet customer needs and desires.

Beyond just product features, this sentiment analysis can also reveal broader trends and themes in consumer behavior and expectations. For example, if there is a sudden increase in positive feedback about a new eco-friendly feature in a product, it may indicate a growing consumer interest in sustainability. Conversely, if a common complaint emerges about the durability of a product, it provides a clear area for improvement. By continuously monitoring and analyzing these sentiments, companies can stay ahead of the curve, adapting quickly to changes in consumer expectations and maintaining a competitive edge in the market.

3. Functionality and Scope

Our project will be structured around four main functionalities:

1. Data Collection: Utilizing a mock Twitter API hosted on our server, we will stream tweets simulating a Twitter API avoiding the cost associated with accessing Twitter's official API, making our project feasible within academic constraints. The tweets will be filtered and the ones related to designated products will be stored in a database for further processing.

2. Sentiment Analysis: We will apply Natural Language Processing (NLP) models to assess whether the sentiments of the selected tweets are sad, joyful, loving, angry, fearful, or surprised by storing the results in a database.

3. Data Visualization: Interactive dashboards will be developed to display sentiment trends and comparisons over time. These visualizations will help in quickly assimilating consumer sentiment and making informed decisions.

4. Workflow Automation: By building pipelines we will automate the workflow to ensure continuous data flow and up-to-date analysis.

4. Architecture

This section goes into the details of the evolution of the architecture covering the differences in scope from the project's first draft. It also provides the alternatives to obtain similar results as well as briefly explaining the benefits of the final architecture.

4.1. Initial Architecture

Figure 1 illustrates the initial architecture diagram for the project. It is separated into 3 blocks: Data Ingestion and Data Storing, Sentiment Analysis, and Visualization and Analytics.

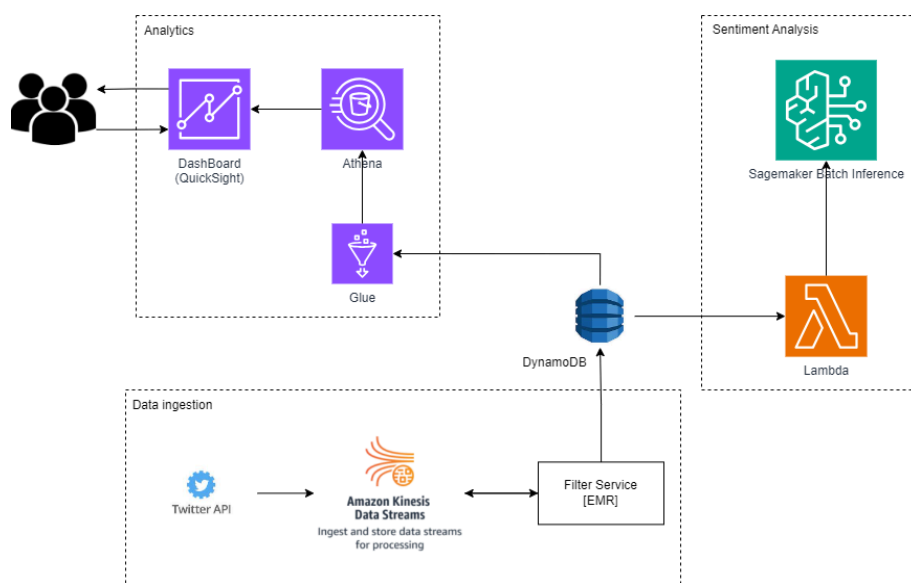


Figure 1. Project Architecture Diagram.

This architecture has the following workflow. We have the Twitter API implemented as a Flask server with a big database of tweets, in order to avoid paying for the actual Twitter API for this proof of concept. The data is retrieved from the Twitter API using Amazon Kinesis Data Streams. Next, to separate the tweets by customer and product an EMR service is deployed with the customer specification. Then the data is stored in DynamoDB. A lambda fetches the data and uploads it to Sagemaker to predict the sentiment of each tweet, which is then stored in DynamoDB with the new information of the sentiment. Finally, the data is processed by Glue and structured through Athena to

provide a smoother ingestion to QuickSight. The user can then interact with the Dashboard through QuickSight to take a look at their analytics.

Throughout the deployment and implementation of the architecture, we have realized that some functionalities could be improved from the architecture point of view. Hence, we are going to detail the different changes for each block.

4.1.1. Evolution of the Data Ingestion

The ingestion pipeline is a part that deviated most from the initial design. We discarded the idea of storing all the filtered tweets in DynamoDB because in the long term, keeping a huge amount of data that will only be read (no update to filtered tweets and no queries) in the database is more expensive than storing the tweets in S3 buckets. For the filtering service, we intended to use AWS EMR, but as we weren't too sure about how to tackle the grouping of tweets by products of a customer, we opted for a more straightforward and flexible solution. About the mock API, we didn't implement an API with Flask and opted for a serverless solution because we didn't want to spend too many resources on a component that would be only used in the POC.

4.1.2. Evolution of the Sentiment Analysis

The sentiment analysis part of the project initially aimed to use SageMaker for model control and retraining. However, the limitations of permissions in the Lab environment as well as the cost of SageMaker made an EC2 instance with a Docker container a better option.

4.1.3. Evolution of the Analytics

From the analytics perspective, the initial idea is maintained. The data is read from the S3 bucket with a Glue job that currently simply parses the date format. Even though this implementation may seem over-dimensioned for its functionality, the structure has been kept for more complex future aggregates.

4.2. Final Architecture

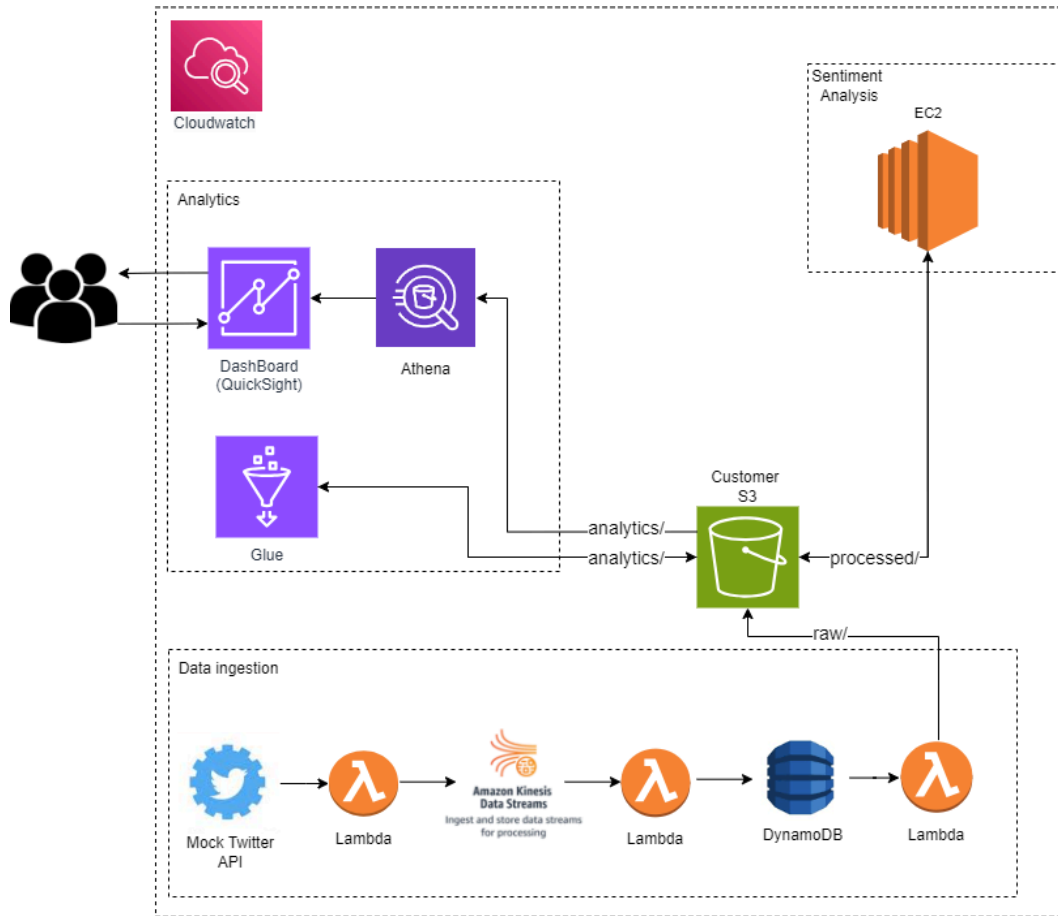


Figure 2. Final architecture.

4.2.1. Data Ingestion

Amazon Kinesis Data Streams: The collected tweets are sent to Kinesis Data Streams, a service designed for handling real-time streaming data. This allows the system to process tweets as they arrive, enabling near real-time processing. This data stream will be produced by a single Lambda and consumed by multiple Lambdas, where all the consumers read the same events (enhanced fan-out, only one shard). We need an ingestion pipeline that quickly filters tweets and stores them, as Twitter generates on average 6k tweets every second.

AWS Lambda: Multiple steps on the ingestion pipeline are implemented as serverless functions with Lambda. With Lambda we have more development flexibility, being able to quickly develop simple logic. Moreover, because none of these functions need to be stateful and aren't running all the time, using a monolithic EC2 or multiple EC2 meant complicating the system without reason.

1. **Mock Twitter API:** Service that returns a random batch of tweets. The tweets are loaded from a predefined [dataset](#) from Kaggle and enriched with random values for gender, age and country.
2. **Kinesis Producer:** Service that gets the latest tweets from Twitter API (in this case Mock Twitter API) and loads them into Kinesis Data Stream.
3. **Keywords Filter:** One keyword filter for each customer. This service reads tweets from the Kinesis Data Stream and filters with the following criteria:

- a. If the tweet text mentions a registered product of the consumer (e.g. iphone for Apple), load the tweet in Dynamodb, in a table with the following name: **<customer>.<product>**.
 - b. If the tweet text mentions a registered product of the consumer and also a feature of the product (e.g. size, color, power), load the tweet in DynamoDB in a table with the following name: **<customer>.<product>.<feature>**.
 - c. Otherwise, discard the tweet.
4. **Batch Loader:** One batch loader for each customer. For each table in DynamoDB associated with the customer (product table or product/feature table), it retrieves all non-processed tweets (tweets that aren't stored in S3 as a JSON batch) and stores them in a JSON batch in S3. These batches are stored in **s3://bucket_customer/product/raw/**.

DynamoDB: Key-value NoSQL database that is used to group the tweets by customer's product or product/feature.

Schema of the tables:

- **tweet_id:** Unique id of the tweet (hash key)
- **creation_date:** Storage date of the tweet in the table (range key)
- **batch_date:** Only defined when the tweet is being batched and loaded to S3.
- **tweet:** JSON encoded as a string of the metadata of the tweet (text, user, age, country).

In the POC, **batch_date** is used for identifying the already processed tweets, so we avoid re-processing them. An additional lambda can be implemented for dropping the processed tweets using **batch_date** or we could directly remove this field and delete the tweets after they are loaded in S3, but it's useful to keep this field in the development stages for the sake of debugging.

S3: Each customer has a bucket for itself. Each one of these buckets serves as a data lake, where all the components of the system read and write. Filtered tweets are stored in **s3://bucket_customer/product/raw/** where the sentiment analysis model can process them. Tweets processed by the sentiment analysis model are stored in the **processed/** directory and the **analytics/** directory is used to store data for the visualization and analytics after passing through our ETL process.

Structure of customer bucket

```
Unset
bucket_customer1:
  - product_1/
    - raw/
      - raw_tweet_batch_1.json
      ...
      - raw_tweet_batch_N.json
    - processed/
      - processed_results_1.parquet
```

```

...
- processed_results_N.parquet
- analytics/
- processed_results_1.parquet
...
- processed_results_N.parquet
...
- product_M.feature_K/
...

```

The buckets are separated by customers for security and resource management reasons. In case of a security breach, this segregation ensures that the impact is contained to the specific customer's data, minimizing the potential damage and protecting the confidentiality and integrity of other customers' information. It also prevents data leaks between customers. This compartmentalization strategy adds an extra layer of security to our data storage architecture, enhancing overall data protection measures and bolstering trust and confidence in our platform's security practices. This separation can also be used as a way to have better control of the resources consumed by each customer.

Alternative designs:

- **Kinesis Firehose instead of DynamoDB with Batch Loaders:** Firehose is a service that allows load streaming data into S3 and other targets (data lakes). At first it seems as the perfect solution for our problem, but we found that Firehose couldn't load batches grouped by a criteria (customer and product/feature). A fix for this was to implement a firehose stream for each product and product/feature for each customer, but maintaining this was pretty expensive.
- **Use AWS EMR with Spark instead of Lambdas with DynamoDB:** Another option was to use AWS EMR with Spark, as they are used extensively for data processing, but we considered that learning Spark from scratch takes a considerable amount of time and we opted for a quick and flexible development with DynamoDB and Lambdas.

4.2.2. Sentiment Analysis

EC2: Amazon EC2 (Elastic Compute Cloud) is a web service that provides secure, resizable compute capacity in the cloud, allowing users to run applications on virtual servers. The EC2 instance retrieves JSON files from the "raw" folder in S3, processes them using the sentiment analysis model, and saves the processed output in the "processed" folder in the same S3 bucket.

S3: Amazon S3 (Simple Storage Service) is a scalable, high-speed, web-based cloud storage service designed for online backup and archiving of data and applications where the raw and processed data is stored.

ECR: Amazon ECR (Elastic Container Registry) is a fully managed Docker container registry that makes it easy for developers to store, manage, and deploy Docker container images securely and at scale. Perfect for storing our docker image with the model.

Sagemaker Batch Inference (not used): Amazon SageMaker Batch Transform is a fully managed service that allows you to perform batch inference on large datasets by using pre-trained machine learning models, enabling scalable and efficient processing of data in bulk. It was thought to be the solution that enabled us to predict the emotion in Twitter data, but after some difficulties on the free account, we decided to use EC2+ECR to manage this part.

4.2.3. Visualization and Analytics

Glue: AWS Glue is a serverless ETL (Extract, Transform, Load) service. In this project, it will be used to transform the tweets and parse the date to timestamp format for the later presented plots.

Athena: Athena is a serverless interactive query service that analyzes data directly in standard SQL without the need to set up or manage any infrastructure. It will serve as the link between AWS Glue and the Dashboard. We have deployed a specific workgroup that manages the stored data in S3 buckets as SQL tables and views for better ingestion to the QuickSight Service.

QuickSight Dashboard: Finally, QuickSight is used to visualize the analyzed data. The dashboard can display various charts and graphs showing trends in sentiment over time, sentiment distribution, and other relevant insights. The idea is to have one dashboard for each product.

5. System Process

5.1. Ingestion Pipeline

Input: Batch of tweets coming from Twitter Mock API.

Output: A json batch of filtered tweets per customer product or customer product with its features.

Process:

1. Twitter Mock API is called each minute by Kinesis producer lambda and gets a batch of latest tweets which we push to the Kinesis data stream.
2. **N** lambdas (one for each customer), read the latest streamed tweets in batches, filter the tweets by product name (and optionally by feature name), and store the filtered tweets in their corresponding DynamoDB table. Each customer has a table for each product and product with features.
 - a. For example, let's say **Customer #1** is interested in monitoring their products "**foo**" and "**bar**", and for "**foo**" its features "**size**" and "**color**". The tweet with the text "**foo color is cool**" will be stored in the DynamoDB table **customer1.foo.color** and **customer1.foo**.
3. DynamoDB is used for grouping the tweets by product name and product/feature names for each customer. When a stored tweet has been finally stored in S3 for further processing, **batch_date** is defined.
4. Every **X** time, **N** lambdas (one for each customer), reads the non-processed tweets from the DynamoDB tables, loads them in batches for each product and stores the json batches in the S3 bucket of the corresponding customer.

- a. Continuing the example, the corresponding lambda reads all the product tables of the corresponding customer searching for non-processed tweets:
 - i. **customer1.foo**: Gets multiple non-processed tweets, including “**foo color is cool**”, loads them in a json batch and stores it in customer S3 bucket (/foo/raw/).
 - ii. **customer1.foo.color**: Like in the previous point, gets non-processed tweets (including example tweet) and stores the batch in customer S3 bucket (/foo/raw/).

Pros:

1. Flexibility: This architecture can be easily changed and extended.
2. Quick deployment: All the ingestion pipeline components are up in less than two minutes.
3. Customer logic separation: Each Kinesis consumer and Dynamodb table consumer owned by each customer can implement its own logic.

Cons:

1. Data duplication: Two customers that are monitoring a product with the same name (edge case), will create two separate tables with the same contents. All the data stored in a product/feature table is also stored in a product table.
2. No true real-time processing: The real-time is broken when the tweets are loaded in DynamoDB in order to obtain the batches of tweets.
3. High dependency on AWS services: In case we change cloud provider, we would have to implement the ingestion pipeline from scratch.

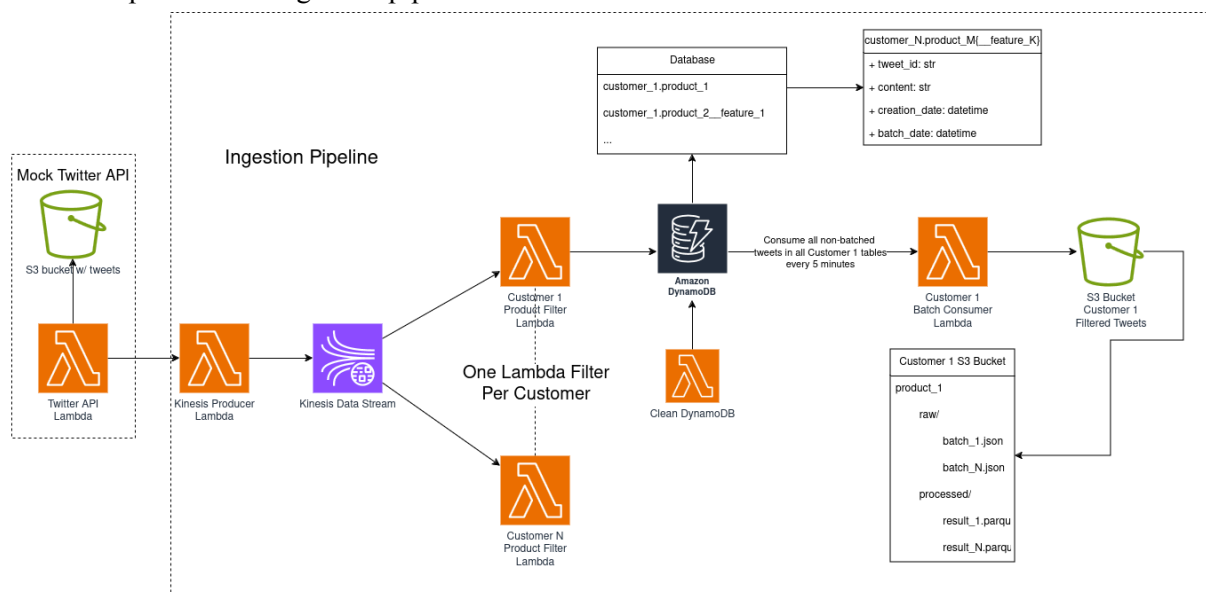


Figure 3.. Ingestion Pipeline Architecture

5.2. Prediction

Input: a batch of tweets classified by product.

Output: Same batch of tweets adding the information for 6 given emotions.

Model:

First of all the Hugging Face model is downloaded from the following repository: `bhadresh-savani/distilbert-base-uncased-emotion`. This model is a transformer based NLP model for sentiment analysis and it has been benchmarked specifically in Twitter data with great performance (F1 of 93.7%). The input of this model is natural language and the response is a vector of probabilities for each of the 6 different emotions (sadness, joy, love, anger, fear, surprise).

Process:

The process for the prediction of the tweets lists all the data that is in the s3 bucket and iterates through it by downloading the JSON into the EC2 sandbox. After that, it predicts the result with the given sentence and saves the output in a pandas DataFrame which is written to s3 with the enriched data in parquet format for good integration with analytics services. As explained in the ingestion part, in s3 the customer data is divided into buckets and the enriched data will be written in the processed prefix. Finally, we delete the raw data to not duplicate the result again as the process is executed every 10 minutes.

All of this process was meant to happen with Sagemaker Batch Inference but after some challenges with the limitations of our free account, we decided to do it with an EC2 instance. This is clearly not optimal in MLOps terms, as Sagemaker provides a very powerful package of MLOps solutions, but as the scope of this project is limited, just a proof of concept, we decided it was a pragmatic solution.

Pros:

1. Cost-Effective for Small Scale: Using an EC2 instance instead of SageMaker can be more cost-effective, especially as we created a small proof of concept with limited resources and budget constraints.
2. Simplified Setup: Setting up an EC2 instance is simpler and quicker compared to configuring SageMaker Batch Inference, which might involve a more complex setup and configuration.
3. Flexibility and Control: We have full control over the EC2 instance, including the ability to install and configure any necessary software or libraries without restrictions.
4. No Service Limits Constraints: Unlike SageMaker, which might have limitations or quotas in a free account, an EC2 instance doesn't have such restrictions, allowing you to avoid issues related to service limits.

Cons:

1. Lack of MLOps Features: EC2 does not offer the integrated MLOps features provided by SageMaker, such as experiment tracking, model versioning, and deployment automation, which are essential for robust machine learning operations.
2. Scalability Issues: Scaling an EC2 instance manually to handle increased load or to distribute workloads can be challenging and require additional setup, whereas SageMaker can handle scaling automatically.
3. Manual Maintenance: Managing and maintaining the EC2 instance, including updates, security patches, and resource management, requires manual effort, whereas SageMaker abstracts much of this overhead.
4. Potential for Higher Long-Term Costs: While EC2 might be cheaper for a proof of concept, costs can escalate if the workload increases, due to the need for larger or multiple instances to handle the load.

5. Operational Complexity: Implementing features like automated retries, logging, monitoring, and alerting requires additional tooling and setup on EC2, whereas SageMaker provides these out of the box.

5.3. Analytics

Input: a batch of tweets with its sentiment prediction by product.

Output: Athena database with the data structured in a table and a precomputed view.

Process:

1. There is a Glue job, that based on a trigger is executed every 3 minutes. This time can be adjusted based on the data load and the prediction execution time, so every time the job is executed there is new data to process. Then, the job for each product of a given customer reads the predictions created in the last 3-minute interval. Next, it parses the date and stores the processed data in the analytics folder of the products of the specific customer.
2. Amazon Athena is built on top of the analytics folder and a table is created for each product of a customer. Additionally, a view precomputes the aggregation of the sentiments of the tweets for each product of a customer. For the POC scope, only one table and one view are created for a given product. These structures facilitate the implementation of the plots and allow to add more aggregations in the future.

Amazon Athena and AWS Glue are commonly used together for analytics projects in AWS due to their seamless integration and powerful capabilities.

Pros:

1. Serverless and Scalable: Being serverless, Athena automatically scales based on the query load, removing the need for infrastructure management. AWS Glue is also serverless and can scale to handle large volumes of data. It manages resources automatically, simplifying ETL (Extract, Transform, Load) operations.
2. Ease of Use: Athena supports standard SQL queries, making it accessible to users familiar with SQL. Glue provides a managed ETL service with a user-friendly interface and built-in transformation capabilities. It also supports Python (PySpark) for more complex transformations.
3. Cost-Effective: In Athena you pay only for the queries you run, based on the amount of data scanned. In Glue, the pricing is based on the number of data processing units (DPUs) used and the duration of jobs, which can be cost-effective for intermittent workloads.
4. Integration with Other AWS Services: Both services integrate well with other AWS services like S3, Redshift, QuickSight, and more, enabling comprehensive data workflows and analytics solutions.

Cons:

1. Performance: While Athena is suitable for many analytical queries, it might not perform as well as a dedicated data warehouse (e.g., Amazon Redshift) for highly complex or large-scale queries. Glue can be slower for large-scale ETL jobs compared to specialized ETL tools or services due to its serverless nature and overhead.
2. Complexity of Advanced Transformations: Although Glue provides various built-in transformations, performing highly complex transformations might require significant customization and expertise in PySpark.

3. Cost Management: Athena costs can become significant if queries are not optimized, as charges are based on data scanned. Inefficient Glue job configurations or long-running jobs can lead to higher costs. So this section can be an advantage or a disadvantage based on the configuration

Other Possible Systems

Amazon Redshift is highly performant for complex queries and large datasets, supports complex joins and aggregations, integrates with AWS services, and has features like Redshift Spectrum to query S3 data directly. However, it requires infrastructure management, can be expensive for small-scale use, and has a steeper learning curve for setup and optimization, and therefore was discarded for this project.

Other options were other cloud providers such as Google BigQuery or Microsoft Azure Synapse Analytics, but due to our knowledge limitations and the project scope, we decided to stay in the AWS environment.

5.4. Visualization

Input: Athena database.

Output: Dashboard with specific visualizations.

Process:

1. First, we create a data source that QuickSight can use to pull data. In our case, this source is Athena.
2. For each product or a product feature a customer has, there will be a corresponding table and view in Athena. Hence, a dataset is configured for each of these sources.
3. Once the data source and the datasets are configured, the different plots can be defined and displayed in a dashboard.

The following diagrams represent an example of a default dashboard that will be provided for a feature/product a client is interested in. **Figure 4** is a pie chart showing the proportion of each emotion in the whole corpus of tweets about the product/feature. In this example anger takes almost half, then joy around one-third with sadness and fear having significantly lesser representation and love and surprise being almost invisible. **Figure 5** is a bar chart with the trend of good (joy, love) and bad (anger, fear, sadness) emotions over time. They can be split by day, month or whichever time period is the most convenient for the use case. **Figures 6 through 8** depict the ratio of good and bad emotions depending on users' gender, country or age.

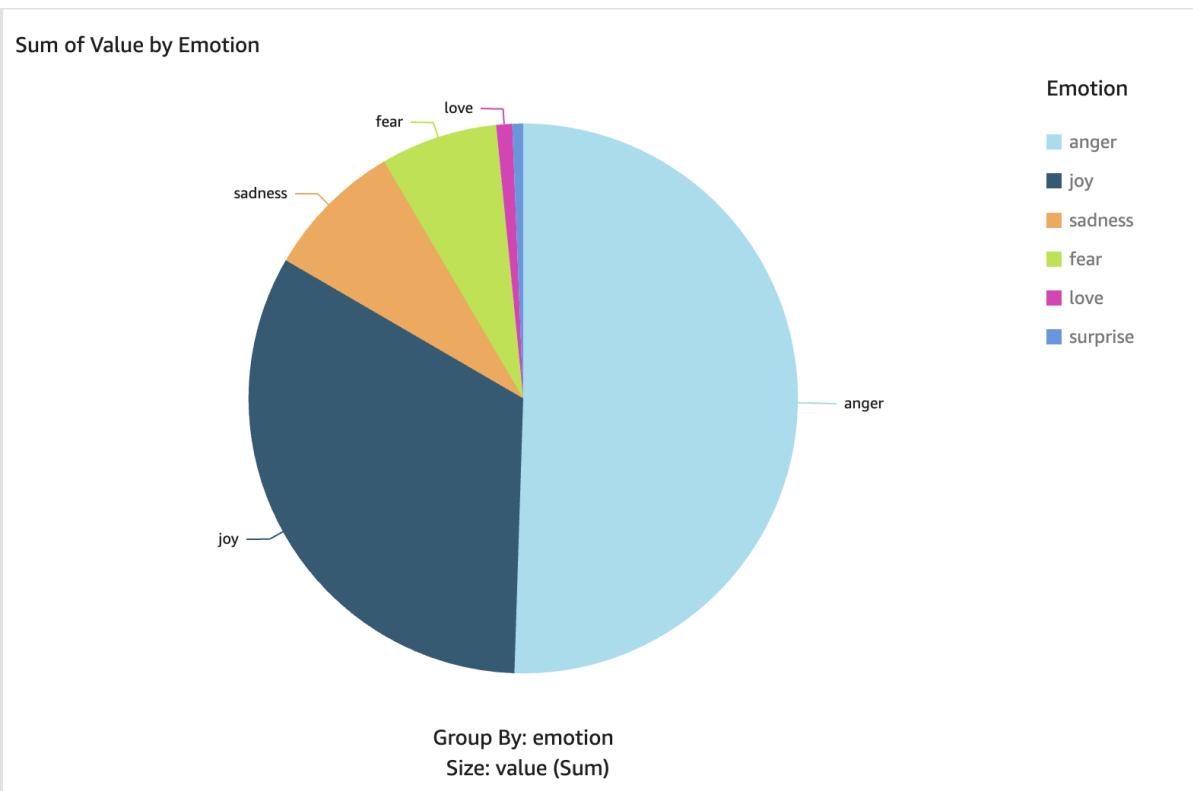


Figure 4. Proportion of all emotions

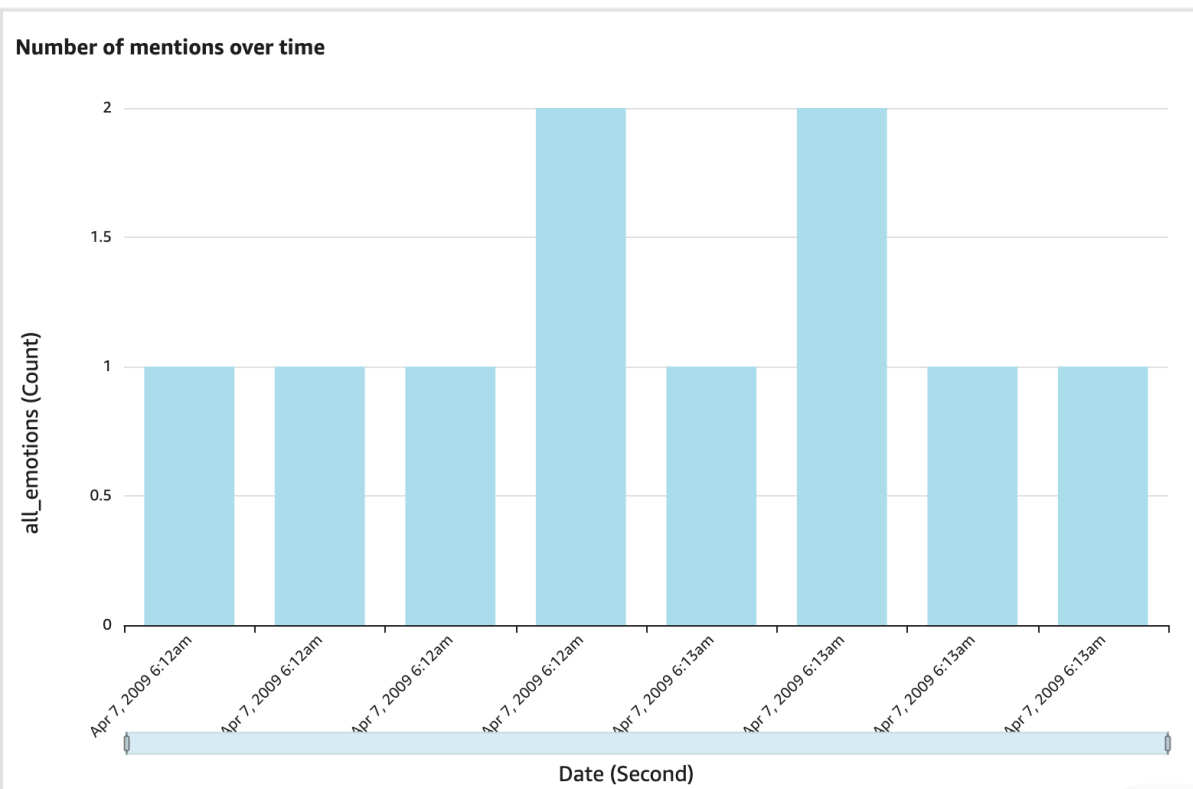


Figure 5. Number of mentions over time.

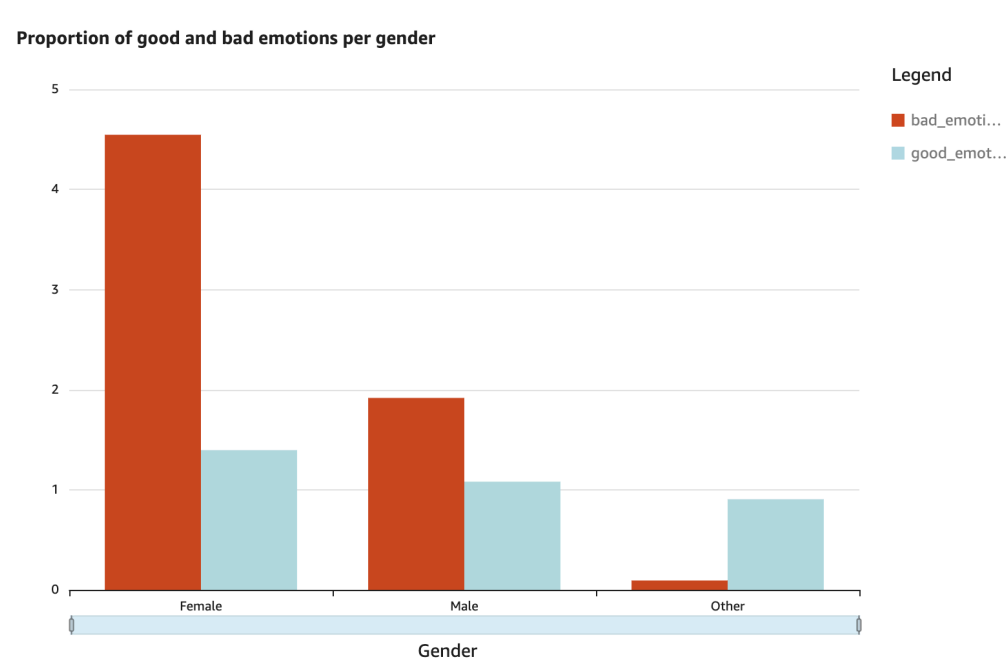


Figure 6. *The proportion of good and bad emotions per gender*

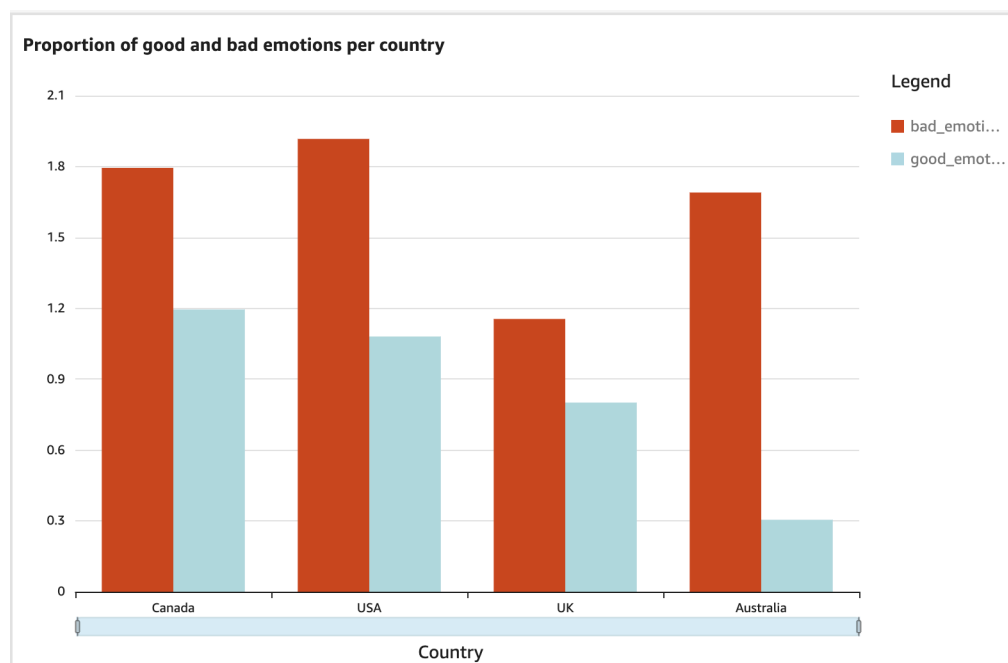


Figure 7. *The proportion of good and bad emotions per country*

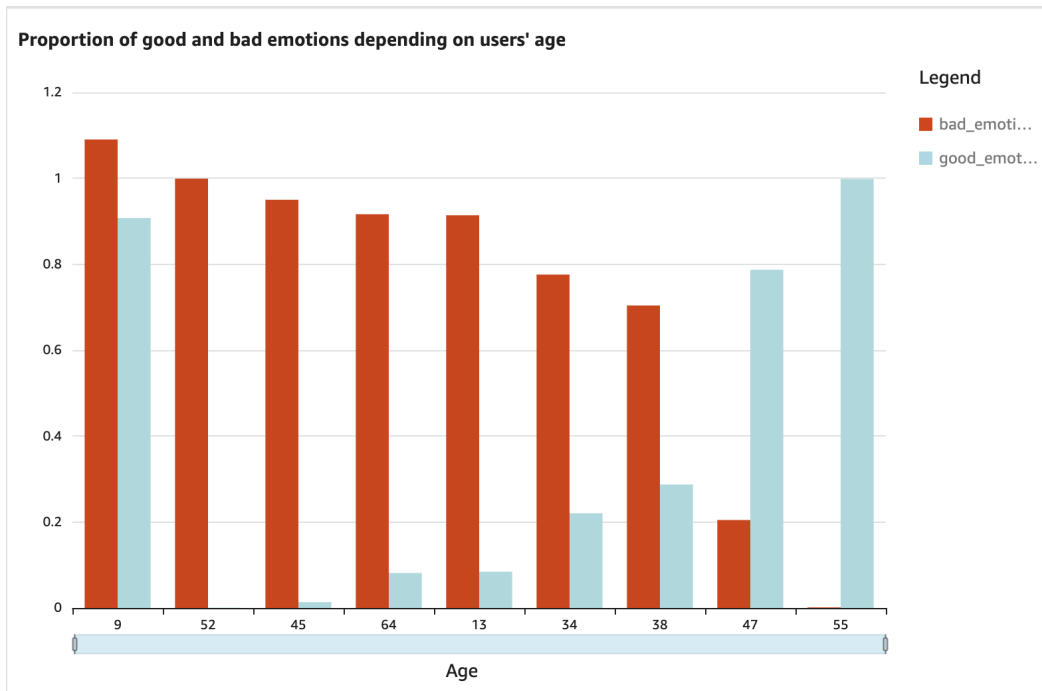


Figure 8. Proportion of good and bad emotions depending on users' age

6. Deployment

The infrastructure for this project has been deployed using Terraform, an open-source infrastructure as code (IaC) tool that enables the efficient and consistent provisioning of cloud resources. By using Terraform, the deployment process becomes automated, reducing the likelihood of human error and ensuring that the infrastructure is reproducible and maintainable. Terraform's code-based configuration allows for version control, making it easy to track and manage changes over time. Additionally, Terraform provides a modular approach, enabling the reuse of configuration files and simplifying updates or scaling of the infrastructure as project requirements evolve.

The Terraform configuration for our project is structured into four distinct modules, each responsible for provisioning specific resources necessary to deploy and manage the infrastructure efficiently.

Common Resources Module: This module handles the creation of common resources, primarily the S3 buckets where the Twitter data is stored. These buckets serve as the central repository for raw, processed, and transformed data, ensuring that all data-related operations are streamlined and accessible.

Data Ingestion Module: This module provisions the resources needed for the ingestion pipeline. Multiple different resources are created in this module: lambdas (mock API, producer, consumers/filters and batch loaders), zip files that stored the code for the lambdas, schedulers for the lambdas, DynamoDB tables for each product, the kinesis data stream, event source mapper (for easily connecting the output of the stream to the consumers/filters).

Data Prediction Module: In this module, an EC2 instance is provisioned to handle sentiment prediction tasks. The instance collects raw Twitter data from the designated S3 bucket and utilizes a preconfigured Docker image containing a Hugging Face model for sentiment analysis that is pulled from ECR. This model predicts the sentiment of each tweet, classifying it with different emotions

(totaling 6). The predicted sentiments are then stored back into the S3 bucket, allowing for further analysis and visualization.

Data Analytics Module: This module provisions resources for data analysis and visualization, including Amazon Athena and Amazon QuickSight. It configures Athena to query the transformed data stored in S3 and integrates with the AWS Glue Data Catalog for seamless metadata management. It also deploys a Glue job that allows the parsing of the Date to the desired format. Additionally, it sets up QuickSight for creating interactive dashboards and visualizations based on the Athena table and view, providing insights into the sentiments and trends within the Twitter data. The QuickSight datasets are deployed through Terraform, but the plots and dashboards are manually created. This allows for more customization and client-specific needs.

By structuring Terraform into these four modules, the deployment process becomes modular, organized, and scalable. Each module focuses on a specific aspect of the infrastructure, making it easier to manage, update, and troubleshoot. This approach enhances the overall efficiency and reliability of the infrastructure, ensuring that each component works harmoniously within the larger system. Additionally, it allowed us to work in parallel and with our different AWS tenants, since we could not create users in the lab environment. We were providing a fast way to share and deploy individually the infrastructure.

7. Use case

The following diagram represents the different ways that the customer can interact with the system. Given a set of provided credentials in QuickSight, the customer logs in through the QuiskSight AWS console. The authentication service is directly managed by the provider, AWS, and we manage the permissions and security groups.

Then the user can access its dashboard, create new ones, and create new graphs and plots. Additionally, he can manage the products that he wants to visualize by creating specific filters or contacting the system administrator to add additional products and features.

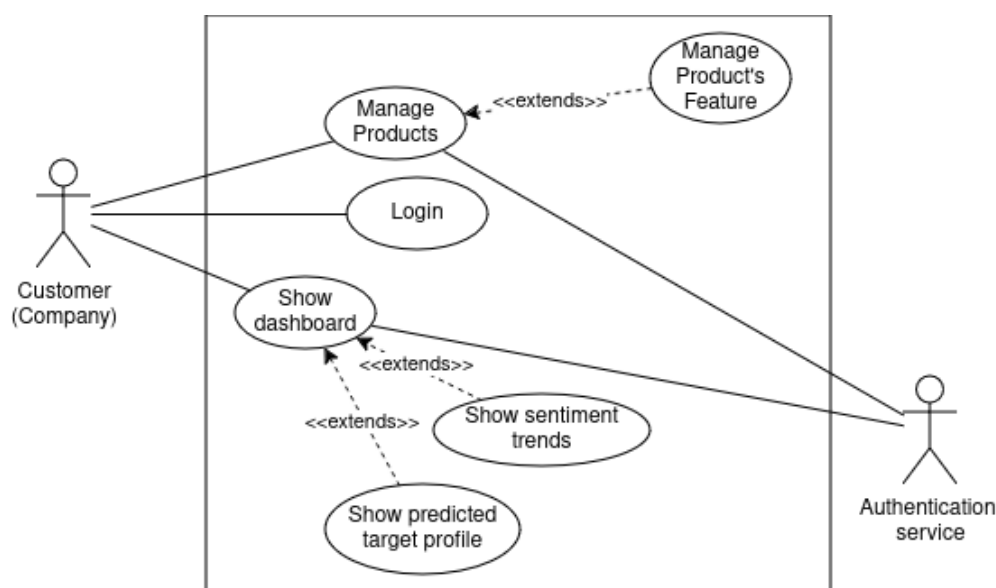


Figure 9. User interaction with the system

8. Project organization

8.1. Division of Responsibilities

Martí was responsible for developing the data ingestion pipeline. His tasks included setting up the Twitter Mock API, designing the ingestion pipeline, implementing Kinesis data streams and event producers with Lambda, creating Lambda Kinesis consumers for filtering data, performing DynamoDB batch grouping, developing a Lambda Batch Loader to S3, and deploying the entire ingestion pipeline using Terraform.

Iker and Ignacio were tasked with evaluating pre-trained sentiment analysis models from Hugging Face and implementing sentiment analysis for the tweets. They also ensured integration with Amazon S3 for seamless data storage and accessibility. They containerized the code with Docker and created an ECR image for deployment. Furthermore, they automated these processes using Terraform for enhanced efficiency and repeatability.

Andreja and Alicia have been collaboratively working on defining various visualizations and different aggregation levels for our analytics dashboards. Their efforts have focused on ensuring that the visual representations are intuitive and informative, catering to diverse analytical needs. They have also developed and deployed the analytics components, ensuring that the data is accurately aggregated and displayed in a manner that supports effective decision-making and strategic planning. The analytics services were automated using Terraform, but due to permit limitations, the dashboards were manually created. In the actual system, there would be default dashboards per product/feature (like the graphs shown in 5.4) and additional ones could be created by the clients themselves or by administrators per clients' request.

We all met up to merge all the parts together in the Terraform code, ensure the workflow worked properly, and tested some edge cases, as not having any data. Then, the final report and the presentation were written and edited by all the team members throughout the project development.

8.2. Meetings and Planning

Once the scope of the project was defined, we met to plan and divide the work as mentioned in the previous section. Since the modules were dependent on the data generated by the initial one, we defined the structure and generated some mock data to allow working in parallel.

Weekly meetings were set to keep track of the project's progress and support any member if needed. Additionally, the members that worked in a specific part together, shared the information more continuously, through chatting platforms. The meetings were mainly online.

Once the different parts were configured and tested, a couple of days were reserved to put all the infrastructure in common and ensure the proper functioning of the workflow.

No specific schedule was set, since most of the members of the team work. Therefore every Monday based on the people's availability the meeting was scheduled for that given week.

8.3. Time Allocation

Martí

1. Meetings - 10 hours
2. Setting up the Twitter Mock API - 4 hours
3. Designing the ingestion pipeline - 8 hours
4. Implementing Kinesis data streams and event producers with Lambda - 8 hours
5. Creating Lambda Kinesis consumers for filtering data - 8 hours
6. Performing DynamoDB batch grouping - 8 hours
7. Developing a Lambda Batch Loader to S3 - 4 hours
8. Deploying the ingestion pipeline using Terraform - 4 hours
9. Merging Deployment - 5 hours
10. Report & Presentation 8 hours

Iker

1. Meetings - 10 hours
2. Evaluating pre-trained sentiment analysis models from Hugging Face - 5 hours
3. Implementing sentiment analysis for the tweets - 8 hours
4. Ensuring integration with Amazon S3 for data storage - 4 hours
5. Merging Deployment - 5 hours
6. Report & Presentation - 8 hours

Ignacio

1. Meetings - 10 hours
2. Ensuring integration with Amazon S3 for data storage - 5 hours
3. Containerizing the code with Docker - 5 hours
4. Creating an ECR image for deployment - 5 hours
5. Merging Deployment - 5 hours
6. Report & Presentation - 8 hours

Andreja

1. Meetings - 10 hours
2. Defining visualizations and aggregation levels for analytics dashboards -10 hours
3. Ensuring data is accurately aggregated and displayed - 2 hours
4. Defining the ETL process in AWS Glue - 7 hours
5. Merging Deployment - 3 hours
6. Report & Presentation - 8 hours

Alicia

1. Meetings - 10 hours
2. Developing and deploying analytics components - 15 hours

3. Ensuring data is accurately aggregated and displayed - 2 hours
4. Merging Deployment - 5 hours
5. Report & Presentation - 8 hours

9. Data Regulation

In Europe, the primary regulation governing the storage and processing of personal data, including age, gender, and nationality, is the General Data Protection Regulation (GDPR). GDPR applies to all organizations operating within the EU, as well as organizations outside the EU that offer goods or services to individuals in the EU or monitor their behavior. It provides a comprehensive framework for data protection, ensuring that personal data is processed lawfully, transparently, and securely.

The General Data Protection Regulation (GDPR) enforces several key principles to ensure the protection of personal data. These principles include lawfulness, fairness, and transparency, requiring that personal data be processed lawfully and transparently with clear information provided to individuals. The purpose limitation principle mandates that data has to be collected only for specific, explicit, and legitimate purposes and not used in ways incompatible with those purposes. Data minimization requires collecting only the data necessary for the intended purpose. The accuracy principle ensures that personal data is accurate and kept up to date. Storage limitation dictates that data should not be stored longer than necessary for identification purposes. Finally, integrity and confidentiality demand that personal data be processed securely, protecting against unauthorized access, loss, destruction, or damage.

Additionally, in Spain, the implementation of the General Data Protection Regulation (GDPR) is primarily governed by the Ley Orgánica 3/2018, de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD). This law complements the GDPR by addressing specific national issues and ensuring the protection of personal data and digital rights within Spain. The LOPDGDD introduces and protects various digital rights, such as the right to digital education, the right to rectification on the internet, and the right to privacy in the use of digital devices in the workplace.

In our project, we prioritize compliance with GDPR principles to ensure responsible handling of data sourced from the Twitter API. We adhere to purpose limitation by strictly defining the objectives for which we collect and process data, ensuring it is utilized solely for our project's intended scope. Additionally, we practice data minimization by selectively gathering only the necessary information from the Twitter API, minimizing the risk of handling excessive or irrelevant data.

We have transparency about our data processing activities, providing clear and comprehensive information on how we collect, process, and utilize Twitter data. We implement robust data security measures to safeguard the integrity and confidentiality of the data, utilizing encryption, access controls, and other security protocols both during transit and storage.

Furthermore, we would establish clear data retention policies to govern the duration for which Twitter data is retained within our project, ensuring that information is not stored for longer than necessary. By upholding these principles, we prioritize the protection of individual privacy rights and uphold responsible data handling practices in accordance with GDPR regulations.

10. Twelve-Factor Methodology

I. Codebase

We have one codebase in a GitHub repository, which ensures that all developers have access to the latest version of the code and facilitates collaboration and version control management.

II. Dependencies

We declare Python dependencies in the *requirements.txt* file to ensure that the required libraries and packages are installed consistently across different environments. Additionally, using Docker for containerization of some services enhances reproducibility and isolation of dependencies.

III. Config

We utilize Terraform variables to facilitate resource sharing among different modules, ensuring modularity and flexibility in our infrastructure setup. While we leverage Terraform for configuration management, some bucket names and IDs are hardcoded in our Python code. Nevertheless, we prioritize security by refraining from hardcoding credentials.

IV. Backing services

Since backing services refer to any external services our application relies on for its normal functioning in our project these include a mock Twitter API, an S3 bucket for binary assets, a memory database, and various AWS products. This approach ensures flexibility and scalability, allowing us to seamlessly swap or add backing services without altering the application's codebase.

V. Build, release, run

While a CI/CD pipeline hasn't been implemented for the Proof of Concept (POC), it's important to consider its benefits, such as automated testing, continuous integration, and deployment automation, especially as the project evolves beyond the POC stage.

VI. Processes

We adopt the stateless architecture, utilizing serverless functions like Lambdas, where each function instance operates independently without retaining the state. Data persistence is achieved through storage solutions like Amazon S3, allowing for scalability and resilience in our application architecture.

VII. Port binding

While the callable services such as lambdas and kinesis data stream are not associated with a port in the traditional sense, they still adhere to the principles of the Twelve-Factor methodology. The docker container inside the EC2 exposes a port for communication allowing external systems to connect to the services running within the container.

VIII. Concurrency

Our system supports scaling by adding more workers or shards across all utilized services. In the case of the instance model, an Auto Scaling Group (ASG) paired with a Load Balancer to handle increased demand and distribute workload efficiently can be implemented.

IX. Disposability

We leverage Terraform for infrastructure management, utilizing its `init`, `plan`, and `apply` commands for swift automated startup, and the `destroy` command for graceful termination when needed.

X. Dev/prod parity

Given the proof of concept (POC) nature of our project, we've opted not to establish distinct development and production environments. Instead, each team member utilizes their own AWS lab environment for testing purposes

XI. Logs

We've configured automatic logging for all our cloud services through CloudWatch, eliminating the need for additional setup or configurations as per the Twelve-Factor methodology's logging guidelines.

XII. Admin processes

At this stage, we haven't implemented specific admin processes as outlined in the Twelve-Factor methodology since our focus is on developing and validating the proof of concept (POC). If we transitioned to a production-ready state, we would integrate the necessary admin processes for managing and maintaining the application in the operational environment.

11. Future Work

In this section, we are going to present possible improvements to our system. First, to efficiently monitor and manage the costs associated with each customer, we will create an administrator dashboard in Amazon QuickSight. This dashboard will aggregate data from various AWS products, which are separated and billed individually for each customer. By displaying these costs on a unified dashboard, administrators can quickly assess financial performance, identify cost-saving opportunities, and allocate resources more effectively. Also, the aggregation of the products can be previously done with glue, so fewer tables have to be managed since all have the same structure.

To ensure data integrity and accessibility for future analysis, we will move the raw data, considered the source of truth, to Amazon S3 for long-term storage. Instead of deleting the data, the data could be stored in S3 Glacier, taking into account and guaranteeing GDPR and LOPDGDD compliance.

Currently, we are duplicating processes and data in our feature sentiment analysis work, particularly for products that have multiple features. The tweets of these products are processed and stored at least once, and as many times as mentioned features per tweet. So instead of duplicating the tweets, we could store the features as an attribute of the tweet in a list format, so the information is not lost and the data is not duplicated.

Additionally, the prediction could be enriched by deploying multiple models and capturing different behaviors. For example, having models with less granularity of predicted emotions, only positiveness and negativeness, and others with a wider range of emotions, which allows us to compare the predictions since it is a domain without ground truth.

This additional information allows us to develop more interesting graphs about the evolution of the sentiments of the people throughout time. Also, Glue and Athena allow creating complex queries, which would be deployed based on the customers' needs.

Another area for improvement is security, due to the lab limitations we could not create different users and security groups based on custom roles and policies. Since we are dealing with sensible data, managing access to the resources is a key point, even though the customers only have access to their specific dashboards. We could implement further encryption methods such as hashing the tweet IDs so data is anonymized.

Finally, and as we have been commenting through the document a big area of improvement is deploying the model on Sagemaker Batch Inference and having a champion vs challenger type scenario when multiple models are tested in order to improve the actual.

12. Conclusion

This project aimed to harness the power of Twitter data for sentiment analysis to provide businesses with actionable insights into consumer feedback on their products. Throughout the development process, we faced various technical challenges, primarily related to the initial ambition of designing a sophisticated data processing system with some IAM role limitations.

Despite the ambitious scope, we successfully designed and implemented a system that performs continuous data ingestion, sentiment prediction, and analysis using mock Twitter data. The system architecture changed a little bit from our initial design because of some constraints and limitations in the AWS account. Not only were we able to work around them but we created a robust solution that can ingest, process, and analyze tweets in small windows. Next, we review the key services of our system.

- **Data Ingestion:** Utilizing AWS services such as Kinesis Data Streams, Lambda functions, and DynamoDB, we created an efficient pipeline for ingesting and filtering tweets based on specific product-related keywords.
- **Sentiment Analysis:** Implementing a pre-trained transformer-based NLP model from Hugging Face, we accurately assessed tweet sentiments, classifying them into six emotions (sadness, joy, love, anger, fear, surprise).
- **Data Visualization:** Developing interactive dashboards using AWS QuickSight, we visualized sentiment trends and comparisons over time, facilitating quick assimilation of consumer sentiment for informed decision-making.

12.1 Challenges and Adaptations

Throughout the project, we encountered several challenges that required us to adapt our initial plans:

- **Ingestion process:** Grouping tweets by customers and products, writing to s3, and achieving true separation between customers and possible products of each customer was really challenging and we were able to do it thanks to DynamoDB.
- **DevOps and MLOps Integration:** The original plan to fully integrate DevOps and MLOps practices was scaled back. Due to the limitations of our AWS free tier account, we opted for more pragmatic solutions such as using EC2 instances instead of SageMaker for batch processing.

- Infrastructure as Code (IaC): We leveraged Terraform to manage our AWS infrastructure, which helped streamline the deployment process and ensure consistency. This approach also provided valuable experience in using IaC for future projects.

12.2 Key Learnings

The project offered numerous learning opportunities, enhancing our understanding of system design, cloud services, and project management:

- System Design and Architecture: We learned the importance of iterative design and the need to remain flexible as project requirements evolve. Our final architecture reflected significant improvements over the initial design, particularly in terms of cost efficiency and scalability.
- AWS Services and Tools: Gaining hands-on experience with AWS services such as Kinesis, Lambda, DynamoDB, S3, EC2, Glue, Athena, and QuickSight, we deepened our understanding of how to leverage these tools effectively in big data projects.
- Project Management: The project highlighted the critical need for clear requirement definitions and communication within the team. This ensures that all members are aligned and understand the project goals and constraints.

In conclusion, we achieved all our initial objectives, the project was a success in terms of the functional system we developed and the knowledge we gained. The practical experience of designing and implementing a complex data processing pipeline using AWS services has equipped us with valuable skills for future projects. We have demonstrated the feasibility of using social media data to provide meaningful business insights, and we are well-positioned to build on this foundation in future work. This project underscores the potential of big data analytics in understanding and responding to consumer sentiment, showcasing how technology can transform raw data into actionable business intelligence